MCP WORKSHOP · APRIL 2026

Building an MCP server with Python.

A companion deck to the hands-on workshop

workshop.md · src/MCP_server.py

What we'll cover.

01 Tools

02 Model Context Protocol

03 References

PART 01 / 03

Tools.

THE FOUNDATION MCP IS BUILT ON

What are tools?

Python functions you make available to an LLM.

You describe each function — name, purpose, parameters — and the LLM decides when to call it.

LLM asks for a call.

Your code runs the function.

LLM gets the result.

The model itself *never* executes code. It only emits a request.

How tool use works.

LLMs generate text. They can't query databases, call APIs, or read files. Tool use is the bridge.

01 **Define tools.**
Tell the LLM what functions are available.

02 **The LLM decides.**
It responds requesting a tool call — it never executes code itself.

03 **Your code executes.**
The client runs the actual function with the LLM's arguments.

04 **Send the result back.**
The client ships the tool result to the LLM.

05 **The LLM answers.**
Now it has real data and can write a natural-language reply.

A brief history.

WHEN	WHO	WHAT
Jun 2023	**OpenAI**	Introduces `function calling` — later renamed *tool use*.
May 2024	**Anthropic**	Ships tool use in the Claude API (general availability).
2024 →	**Everyone else**	Google (Gemini), Mistral, Cohere, and others follow with their own variants.

Models are fine-tuned to detect when a function is needed and to emit JSON matching the declared signature.

No cross-provider standard.

Every provider ships its own API format for defining tools and handling tool calls.

JSON Schema for parameters is broadly shared — message shapes, result envelopes, and field names are not.

DIFFERENCES ACROSS PROVIDERS

`param schema`	mostly JSON Schema ✓
`message format`	different
`tool result`	different
`role names`	different
`errors`	different

One of the motivations behind MCP: tool use lets an LLM call functions, but every provider does it differently. **MCP standardizes the layer above.**

PART 02 / 03

Model Context Protocol.

AN OPEN STANDARD FOR TOOLS AND CONTEXT

What is MCP?

An open protocol that standardizes how applications provide **tools** and **context** to Large Language Models.

Announced by Anthropic, November 2024 · adopted by other vendors since

A common language between tool providers and LLM clients

Why MCP exists.

THE PROBLEM

Every provider has its own tool-use format.

Every app that wants to expose tools re-implements the plumbing for each one.

THE MOVE

Separate *who provides the tools* from *which LLM consumes them*.

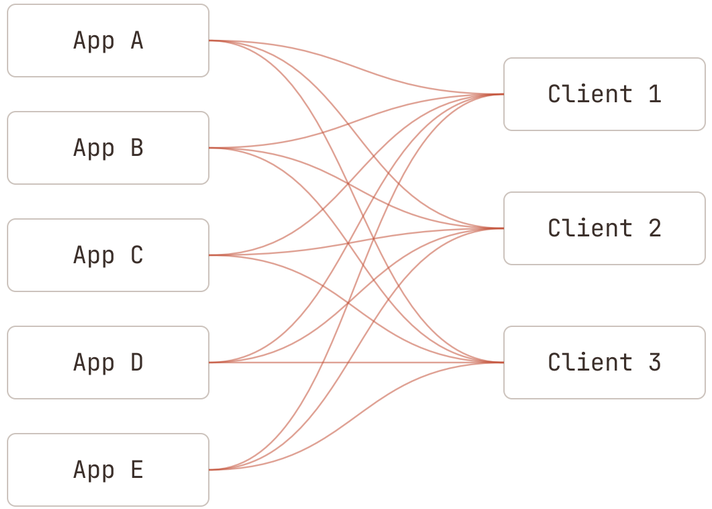
Tool authors integrate once. Clients implement the protocol once. Everything else connects through the standard.

The N×M problem.

N apps × M clients. Without a standard, every pair is a bespoke integration.

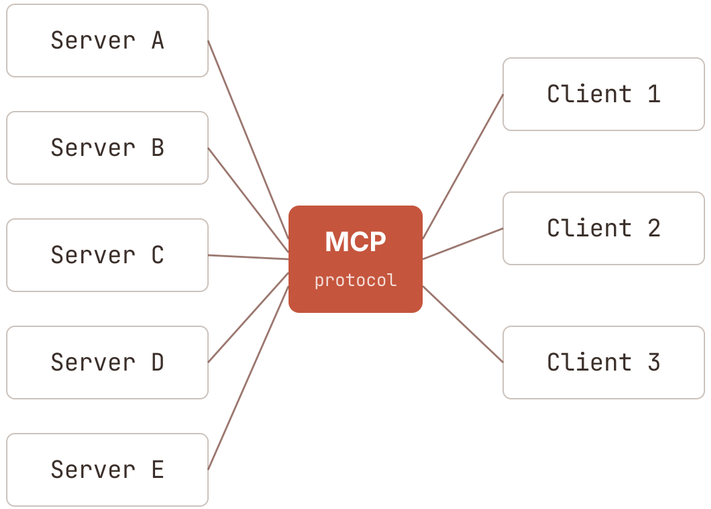
Without a standard

N × M custom integrations



With MCP

N + M · each side speaks the protocol



What MCP adds over raw tool use.

Reusability

One MCP server works with any MCP-compatible client — Claude Desktop, Claude Code, IDE extensions.

Decoupling

Tool authors don't need to know which LLM will consume them.

More than tools

MCP also standardizes *resources* (read-only data) and *prompts* (reusable templates).

Transport flexibility

The same server runs over stdio (local subprocess) or streamable HTTP (remote).

Server vs client.

ROLE · EXPOSES

Server

Exposes tools, resources, and prompts. This is what we build today.

FILE

`src/MCP_server.py`

ROLE · CONSUMES

Client

Connects to one or more servers, discovers what they expose, hands it to the LLM.

EXAMPLES

`Claude Code · Claude Desktop`
`IDE integrations · src/MCP_client.py`

Transports.

Same server code. Two ways to reach it — only the startup argument changes.

TRANSPORT · LOCAL

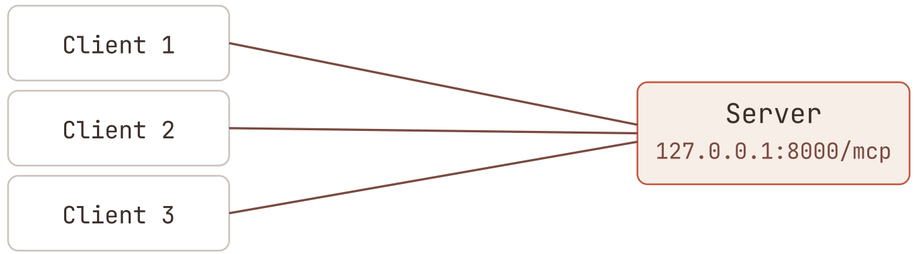
stdio

The client launches the server as a subprocess. They communicate over stdin and stdout. Simple, local only.

TRANSPORT · REMOTE

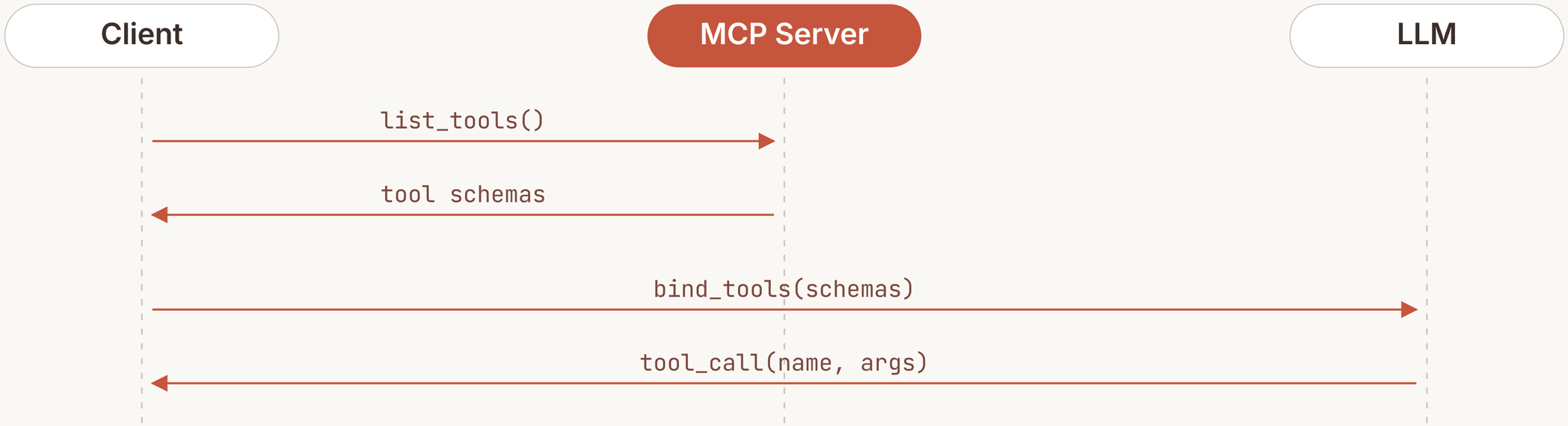
streamable-http

The server runs as its own process. Clients connect over a URL. Enables remote servers shared across clients.



Runtime tool discovery.

The client doesn't import tools. It asks the server what's available — every time it connects.



Consequence: tools can be added or changed on the server without touching the client.

FastMCP, the Python SDK.

High-level class from the official MCP SDK. Schemas are generated from your function signature and docstring.

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("agenda")

@mcp.tool()
def get_talks_by_day(day: str) -> list[dict]:
    """Return every talk scheduled on the given day."""
    return db.query(day=day)
```

Python TypeScript Kotlin Swift ...and more

PART 03 / 03

References.

WHERE TO READ MORE

Documentation and resources.

[1] OpenAI — Function calling announcement (Jun 2023)
openai.com/index/function-calling-and-other-api-updates

[2] Anthropic — Tool use GA announcement (May 2024)
anthropic.com/news/tool-use-ga

[3] Anthropic — MCP announcement (Nov 2024)
anthropic.com/news/model-context-protocol

→ MCP Python SDK — FastMCP quickstart
github.com/modelcontextprotocol/python-sdk

→ Official MCP docs
modelcontextprotocol.io/docs/sdk

→ DeepLearning.AI — Build Rich-Context AI Apps with Anthropic
learn.deeplearning.ai/courses/mcp-build-rich-context-ai-apps-with-anthropic

END OF DECK

Over to the workshop.

WORKSHOP.MD · QUESTIONS WELCOME